IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS

SOFTWARE SYSTEMS

Friday, 8 May 2026, 2:30 PM

Time allowed: 2:00 hours

There are THREE questions on this paper.

Answer ALL questions.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : S. Baig
Second Marker(s) : J. Pitt

© Imperial College London

ELEC50014 Software Systems
Final Examination

Time: 2 hours Marks: 60

Question 1 [20]

(a) For each of the following statements, indicate whether it is TRUE or FALSE and justify your answer briefly.

i. Consider a process on a UNIX-based system which can be in one of the following states at any time: new, ready, running, waiting, or terminated. The process may enter the waiting state from either the running state or the new state.

ii. If a system call does not block, then the calling process cannot be context-switched until control returns to user mode.

iii. Consider a process that traverses a large array of integers and performs no I/O or system calls. The process can enter the kernel only due to timer interrupts.

iv. On a UNIX-based system, for multiple processes to gain shared access to a file through memory mapping, the file needs to be mapped to the same virtual address in each process's virtual address space.

v. On a UNIX-based system, it is possible for a child process to share the same physical code segment pages as its parent.

vi. A system uses Round-Robin scheduling with time quantum q. After upgrading to a faster processor, reducing q may not necessarily reduce the average response time of ready processes.

vii. In a Multi-Level Feedback Queue (MLFQ) scheduler, once a job in the highest priority queue starts running, then it runs to completion before any job in a lower-priority queue is scheduled.

viii. In the Linux Completely Fair Scheduler (CFS), the principle of proportional share scheduling is implemented by ensuring that all ready processes receive an equal share of CPU time within a fixed time window called `sched_latency`.

$[8 \times 2 = 16]$

(b) Table 1 below shows the arrival and burst times (in seconds) of three processes:

Process	Arrival Time (s)	CPU Burst Time (s)
A	0	7
B	2	2
C	4	5

Table 1: Process arrival and CPU burst times

i. Compute the Average Turnaround Time (ATT) for each of the following scheduling policies: First-Come First-Served (FCFS), non-preemptive Shortest Job First (SJF), and Shortest Time to Completion First (STCF).

ii. If Round Robin (RR) scheduling is used with a time quantum of 1 second, would the ATT increase or decrease relative to the policies above? Briefly justify your answer (no computation required).

[4]

Question 2 [20]

Consider a multithreaded server with a shared database. The database can reliably handle at most N concurrent clients. The server must implement synchronisation such that client threads acquire a slot before accessing the database and release it afterwards, and the number of clients accessing the database at the same time never exceeds N.

(a) In this part, design a synchronisation solution using semaphores. Listing 1 shows the outline of the database usage without synchronisation included.

i. Add appropriate semaphore-based synchronisation to the code in Listing 1 so that it works as expected. For simplicity, assume that a globally created and initialised semaphore called `sem` is available for use.

ii. Show how the semaphore will be initialised before its use.

For your reference, Table 2 shows the three main semaphore operations with brief descriptions.

```
void *access_db_reliably(void *arg)
{
    //thread enters the db below
    access_db(arg);
    return NULL;
}
```

Listing 1: Database usage without synchronisation

Function	**Description**
`int sem_init(sem_t *sem, int pshared, unsigned value);`	Initialise semaphore
`int sem_wait(sem_t *sem);`	Acquire/wait
`int sem_post(sem_t *sem);`	Release/signal

Table 2: Semaphore operations with brief descriptions

[3]

(b) Can this synchronisation problem be solved using only mutexes? We still want threads to sleep if no slots are available. Any number of mutexes may be used, but no other synchronisation or atomic primitives can be used. State, with justification, whether this is Possible or Not Possible.

[3]

(c) In this part, the aim is to implement the synchronisation using busy-waiting with atomic primitives. The code in Listing 2 shows how two functions: `acquire_slot()` and `release_slot()` are used to reliably access the database:

```
void *access_db_reliably(void *arg)
{
    acquire_slot();
    access_db(arg);
    release_slot();
    return NULL;
}
```

Listing 2: Usage of acquire and release functions

The goal is to implement the `acquire_slot()` and `release_slot()` functions using atomic `compare_and_swap()` and `fetch_and_add()` operations at appropriate places. The skeletons of these two functions are provided in Listing 3. Complete these functions so that they work correctly overall. Assume that `permits` is a global integer variable initially set to N. For your reference, the definitions of `compare_and_swap()` and `fetch_and_add()` have been provided in Table 3.

```
void acquire_slot()
{

  int expected = permits;

  // Write this code

}
```

```
void release_slot()
{


  // Write this code


}
```

Listing 3: Skeleton code for acquire and release functions

Function	**Description**
`int compare_and_swap(int *ptr, int expected, int new_val);`	Atomic conditional update
`int fetch_and_add(int *ptr);`	Atomic increment

Table 3: Atomic operations with brief descriptions

[6]

(d) The database requirements have changed such that each client thread must now acquire two slots before accessing the database. The total number of available slots remains N. We decide to reuse the synchronisation solution from part (c), which results in the code shown in Listing 4. State, with justification, whether the synchronisation solution in Listing 4 is Correct or Incorrect.

```
void *access_db_reliably(void *arg)
{
    acquire_slot();
    acquire_slot();
    access_db(arg);
    release_slot();
    release_slot();
    return NULL;
}
```

Listing 4: Modified database access requiring two slots

[4]

(e) The solutions implemented in parts (a) and (c) do not ensure fairness. Specifically, requests that arrive later may get to use the database before the requests that came earlier. Suppose we wanted to implement fairness. Indicate which of the following options (either only one of them, both of them, or none of them) is viable to implement a fair solution. Provide justification for your answer.

Option 1: A solution using a condition variable with a mutex, and user-level logic.

Option 2: A solution using a ticket lock implementation with atomic primitives, and user-level logic.

[4]

Question 3 [20]

(a) Consider a system that uses multi-level paging for address translation. The system uses 42-bit virtual addresses and contains 4 GB of physical memory. The page size is 4 KB. Each page table entry is 4 bytes long. Each individual page table at every level must fit within a single page frame.

i. How many bits of the virtual address are required for the page offset and for the virtual page number, respectively?

[2]

ii. What is the width (in bits) of the page frame number?

[2]

iii. If all page table entries were stored in a single linear page table, how many pages would be required to store the entire table?

[2]

iv. What is the minimum number of page-table levels required for address translation in this system?

[2]

v. Suppose a particular process uses only 128 MB of virtual memory, with virtual addresses ranging from 0 to $2^{27} - 1$. How many page-table pages are required at each level to translate this process's address space?

[4]

(b) Consider the following statement: "Page tables are typically pinned to kernel memory and are not swapped out, even under severe memory pressure." State, with justification, two possible reasons behind this design choice.

[4]

(c) For each factor below, indicate whether it will always result in fewer TLB misses during address translation or not. For each case, state your answer as Always or Not always, and justify the answer.

i. Larger virtual page sizes

ii. Smaller virtual address space size

iii. Increased TLB size

[4]